
[Re] Diffusion Posterior Sampling for Noisy Inverse Problems

Dan Vicente Ihanus
KTH Royal Institute of Technology
danvi@kth.se

Alexander Gutell
KTH Royal Institute of Technology
agutell@kth.se

Ludvig Skare
KTH Royal Institute of Technology
lskare@kth.se

Abstract

This paper aims to reproduce the experiments presented in Chung et al. [2023], where the authors propose a sampling technique for solving inverse problems with diffusion models as priors. Our re-implementation successfully reproduces the results to a satisfactory extent based on the information provided in the original paper, though with worse FID, LPIPS, SSIM, and PSNR scores compared to the reported benchmarks. Additionally, we extend the original implementation by incorporating a fast sampling algorithm for diffusion models, DPM-Solver Lu et al. [2024]. Further, we conduct several additional experiments, including ablation studies on mask size for Random and Box Inpainting, and perform an in-depth statistical analysis of LPIPS scores to better understand the model’s performance and limitations.

Code is available at https://github.com/KTH-DD2412-Project-Group-18/diffusion_posterior_sampling

1 Introduction

The paper *Diffusion Posterior Sampling for General Noisy Inverse Problem* Chung et al. [2023] presents a novel way to solve nonlinear inverse problems using diffusion models as priors. The general setting of inverse problems (with additive noise) is that one has observations,

$$y = \mathcal{A}(x) + n, \quad (1)$$

for some nonlinear measurement-operator $\mathcal{A}$ and some "noise" n . The goal is to recover x given these noisy (possibly partial) observations y . In particular, one wants to estimate $p(x|y)$. By Bayes’ theorem, this posterior distribution can conveniently be written as

$$p(x|y) = \frac{p(y|x)p(x)}{p(y)}.$$

Using diffusion models as priors for the inverse problem stems from the following observation,

$$\begin{aligned} \nabla_{x_t} \log p_t(x_t|y) &= \nabla_{x_t} \log \left(\frac{p_t(y|x_t)p_t(x_t)}{p(y)} \right) \\ &= \nabla_{x_t} \log p_t(y|x_t) + \underbrace{\nabla_{x_t} \log p_t(x_t)}_{\approx s_\theta(x_t, t)}. \end{aligned}$$

Here $s_\theta(x_t, t)$ is known as the (unconditional) score-function and is obtained from a trained diffusion model. The first term, however needs to be estimated using some knowledge about the nature of the inverse problem.

2 Main Contribution of Original Paper

The main idea of the paper Chung et al. [2023] is that one can apply the approximation,

$$p(y|x_t) \approx p(y|\mathbb{E}[x_0|x_t]) =: p(y|\hat{x}_0), \quad (2)$$

and use Tweedie’s-formula to leverage diffusion models for further approximating

$$\mathbb{E}[x_0|x_t] = \frac{1}{\sqrt{\alpha_t}}(x_t + (1 - \bar{\alpha}_t)\nabla_{x_t} \log p_t(x_t)) \approx \frac{1}{\sqrt{\alpha_t}}(x_t + (1 - \alpha_t)s_\theta(x, t)). \quad (3)$$

The approximation 2 is motivated by the so-called Jensen gap (see Theorem 1 here). But a more recent paper argues that sampling from the posterior in this way can, in fact, be seen as being closer to finding the MAP estimate. Anonymous [2024].

The main result of the paper is the following proposition: If one has a diffusion model that is reasonably good at predicting noise, along with a well-defined measurement operator, one can solve (almost) any kind of inverse problem of the form 1, by sampling from the posterior using diffusion priors.

The authors leverage tractable distributions (Gaussian/Poisson) to get explicit expressions of the posterior distribution $p(y|x_t)$. We note that in the case of Gaussian noise $n \sim \mathcal{N}(0, \sigma^2 I)$, we obtain $y|\hat{x}_0 \sim \mathcal{N}(\mathcal{A}(\hat{x}_0), \sigma^2 I)$, and in the case of Poisson noise, $y|\hat{x}_0 \sim \text{Poisson}(\mathcal{A}(\hat{x}_0))$. If one further assumes that $\mathcal{A}(x_0) \gg 0$, then it is possible to approximate the Poisson conditional by $y_j|\hat{x}_0 \sim \mathcal{N}(\mathcal{A}(\hat{x}_0), y_j)$. Sampling from the posterior is done by noting that for $n \sim \mathcal{N}(0, \sigma^2 I)$

$$\nabla_{x_t} \log p_t(x_t|y) \approx s_\theta(x_t, t) - \rho \nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0)\|_2^2,$$

and for $n \sim \text{Poisson}(\mathcal{A}(\hat{x}_0))$,

$$\nabla_{x_t} \log p_t(x_t|y) \approx s_\theta(x_t, t) - \rho \nabla_{x_t} \sum_{j=1}^n \frac{(y_j - [\mathcal{A}(\hat{x}_0)]_j)^2}{y_j},$$

by applying the Poisson-normal approximation. An important computational aspect to consider in the DPS algorithm is that by computing the gradient of the loss, we are in fact performing backpropagation through the entire score network due to the chain rule. Indeed,

$$\nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0)\|_2^2 = -\frac{2}{\sqrt{\alpha_t}}(y - \mathcal{A}(\hat{x}_0))\mathcal{A}'(\hat{x}_0)(1 + (1 - \alpha_t)\nabla_{x_t} s_\theta(x_t, t)).$$

This makes the DPS algorithm scale as $\mathcal{O}(NFE \times c(s_\theta))$, with $c(s_\theta)$ being the cost of backpropagation through the network and NFE (Neural Function Evaluations) typically ranges from $10^1 - 10^3$. The full details of the methods are seen in Algorithms 1 & 2. This computational intensity has some practical implications. In particular, one needs *at least* one GPU for computations as well as large enough memory for storage of network parameters and intermediate computations needed for autograd. Considering the size of modern networks, this effectively sets some real-world constraints on the method’s practical viability.

Algorithm 1 DPS - Gaussian	Algorithm 2 DPS - Poisson
Require: $N, y, \{\zeta_i\}_{i=1}^N, \{\bar{\sigma}_i\}_{i=1}^N$ 1: $x_N \sim \mathcal{N}(0, I)$ 2: for $i = N - 1$ to 0 do 3: $\hat{s} \leftarrow s_\theta(x_i, i)$ 4: $\hat{x}_0 \leftarrow \frac{1}{\sqrt{\alpha_i}}(x_i + (1 - \bar{\alpha}_i)\hat{s})$ 5: $z \sim \mathcal{N}(0, I)$ 6: $x'_{i-1} \leftarrow \frac{\sqrt{\alpha_i(1-\bar{\alpha}_{i-1})}}{1-\bar{\alpha}_i}x_i + \frac{\sqrt{\alpha_{i-1}\beta_i}}{1-\bar{\alpha}_i}\hat{x}_0 + \bar{\sigma}_iz$ 7: $x_{i-1} \leftarrow x'_{i-1} - \zeta_i \nabla_{x_i} \ y - \mathcal{A}(\hat{x}_0)\ _2^2$ 8: end for 9: return $\hat{x}_0$	Require: $N, y, \{\zeta_i\}_{i=1}^N, \{\bar{\sigma}_i\}_{i=1}^N$ 1: $x_N \sim \mathcal{N}(0, I)$ 2: for $i = N - 1$ to 0 do 3: $\hat{s} \leftarrow s_\theta(x_i, i)$ 4: $\hat{x}_0 \leftarrow \frac{1}{\sqrt{\alpha_i}}(x_i + (1 - \bar{\alpha}_i)\hat{s})$ 5: $z \sim \mathcal{N}(0, I)$ 6: $x'_{i-1} \leftarrow \frac{\sqrt{\alpha_i(1-\bar{\alpha}_{i-1})}}{1-\bar{\alpha}_i}x_i + \frac{\sqrt{\alpha_{i-1}\beta_i}}{1-\bar{\alpha}_i}\hat{x}_0 + \bar{\sigma}_iz$ 7: $x_{i-1} \leftarrow x'_{i-1} - \zeta_i \nabla_{x_t} \sum_{j=1}^n \frac{(y_j - [\mathcal{A}(\hat{x}_0)]_j)^2}{y_j}$ 8: end for 9: return $\hat{x}_0$

We re-implement Algorithms 1 & 2 and the experiments indicated in the paper Chung et al. [2023] on the ImageNet and FFHQ datasets as best as we can, with the information provided in the original paper. Even though the authors provide their own public repository, we have chosen to replicate the algorithm from scratch to better understand what assumptions are made in order for the model to work in practice and to possibly detect if any part of the implementation is critical to achieve the results the authors present.

We find that our implementation achieves results that are close to the ones presented in the original paper, but in general slightly worse. This is particularly evident in our implementation of the Fourier magnitude (see Appendix B).

3 Scope of reproducibility

3.1 Main Experiments

We re-implemented the denoising procedure for six distinct types of measurement models: inpainting, bicubic downsampling (low resolution), Gaussian blur kernel, motion blur, Fourier magnitude measurement, and nonlinear deblurring. These operators were applied to images from the ImageNet 256x256, FFHQ 256x256 datasets. The original paper employed two separate models as scoring functions, one for each dataset, and we included both in our implementation. The following key claims were identified in the original paper:

Claim 1 The proposed method achieves notably low FID, LPIPS, SSIM and PSNR scores when addressing linear inverse problems (on both datasets), including: *Super-resolution ($\times 4$)*, *Inpainting (box)*, *Inpainting (random)*, *Deblurring (Gaussian blur)*, *Deblurring (motion blur)*

Claim 2 Similar to Claim 1, the method achieves notably low FID and LPIPS scores for *nonlinear deblurring* and *phase retrieval* for the FFHQ dataset.

Considering the claims above, this report tries to address the following questions

- Can we build the measurement operators using the authors descriptions of them?
- Can we reproduce the FID/LPIPS/SSIM/PSNR scores presented in the original paper, using the information provided in the paper?
- What missing information for the experiments/implementation might affect the results significantly?
- What assumptions are made by the authors to achieve the results?

3.2 Additional Contributions

We extend the methods in the paper by considering an additional dataset, namely CelebA-HQ 256×256 dataset. The motivation for this is that the pretrained score-model from Choi et al. [2021] has been trained on the full FFHQ-dataset, which might make the results slightly misleading. We assume that the samples in the CelebA-HQ dataset has similar characteristics, but are unseen by the pretrained score network. Secondly, we build more measurement models than used in their experiments (See section 4.1 for a full description of these). Thirdly, we present additional statistics (min, max, mean, std) on the evaluation metrics. Fourthly, we perform a small ablation study on the amount of masked pixels in the inpainting measurement-operators, to try and answer the question of how much missing data the DPS-algorithm is able to fill in. Finally, we lay some groundwork for integrating the DPS algorithm with fast, higher-order diffusion sampling using the multistep DPM-Solver Lu et al. [2024].

4 Methodology

The DPS algorithm, measurement operators and simulations outlined in Chung et al. [2023] were re-implemented based on the explanations in the paper. We use two pretrained score functions, one which has been trained on samples from ImageNet and a slightly smaller network which has been trained on FFHQ,

- *ImageNet* - UNet architecture from OpenAI [2024], trained with standard DDPM-training.

- *FFHQ* - UNet architecture from Choi et al. [2021] trained with standard DDPM-training.

Both of the codebases for the score-functions above build upon the guided/improved-diffusion repository of openAI OpenAI [2024] and thus our implementation does as well.

4.1 Measurement Operators

We re-implement the measurement operators from the paper independently. The only exceptions are the operators the authors themselves state they obtain from external sources. The operators considered in the original paper, a brief description of them and their sources are summarized below

- *Random inpainting* - Mask a random set of pixels of image by multiplying with zero.
- *Box inpainting* - Mask a square box of pixels in image by multiplying with zero.
- *Gaussian blur* - Blur image with Gaussian convolution kernel of size 61×61 .
- *Super resolution* - Downsample image using bicubic sampling and upsample to original size using nearest-neighbor upsampling
- *Fourier phase retrieval* - Compute discrete Fourier transform of padded image and compute magnitude. This is equivalent to oversampling in the Fourier domain III [2007].
- *Motion blur* - Compute motion blurring of image using convolution kernels obtained from Borodenko [2024].
- *Nonlinear Blur* - Compute nonlinear blurring of image using kernels obtained from Tran et al. [2021]

We also include two extra measurement operators in our repository, which we summarize below

- *Magnitude* - Take the absolute value of image, in essence computing the negative.
- *Grayscale* - Compute the black-and-white version of image.

We note that we have not been able to run experiments using these two extra measurement operators.

4.2 Parameter Search

The paper sets the step-size in the DPS-step to $\zeta_i = \eta / \|y - \mathcal{A}(\hat{x}_0)\|$ for constant values $\eta > 0$, depending on which measurement operator they consider. However, we note that using the values they propose for η yields rather bad results. So in order to find good step-sizes, we performed hyperparameter search. The overall strategy here was to do a coarse search and a subsequent fine-grained search. The coarse hyperparameter search was conducted similarly for all measurement operators and datasets with the following 11 step sizes: [1 5 10 15 20 25 30 35 40 45 50]. The finer hyperparameter search was conducted with 9 step sizes centered around the previous best value. The finer search was conducted with increments of 0.5. These parameter searches were conducted for each measurement for each dataset resulting in a total of 30 hyperparameters searches for Gaussian noise and 6 hyperparameter searches for Poisson noise.

The final choice of step size was based on a qualitative assessment, as the presence of multiple metrics introduced ambiguity regarding which metric should be prioritized for optimization. This qualitative assessment was based on generating the same single image for each learning rate being tested.

Table 1: step sizes used for different methods and datasets after parameter search.

Dataset	Inpaint (random)	Inpaint (box)	SR ($\times 4$)	Deblur (motion)	Deblur (gauss)
ImageNet	40	0.45	0.4	0.25	0.3
FFHQ	40	0.5	0.4	0.15	0.3
CelebA	40	0.5	0.4	0.15	0.3

4.3 Evaluation

All metrics were calculated by external code: FID Seitzer [2020], LPIPS Zhang et al. [2018], SSIM Team [2024b], and PSNR Team [2024a]. Each metric was taken over 250 images. The backbone model in for LPIPS was set to "vgg".

5 Results

Table 2: Quantitative evaluation (FID, LPIPS, SSIM, PSNR) of solving linear inverse problems on ImageNet 256×256-1k validation dataset. Noise model: Gaussian

Method	Inpaint (random)		Inpaint (box)		SR ($\times 4$)		Deblur (motion)		Deblur (gauss)	
	Ours	Paper	Ours	Paper	Ours	Paper	Ours	Paper	Ours	Paper
FID ↓	69.91	35.87	136.93	38.82	55.50	50.66	164.70	56.08	88.83	62.72
LPIPS ↓	0.361	0.303	0.503	0.262	0.324	0.337	0.586	0.389	0.482	0.444
SSIM ↑	0.526	0.739	0.487	0.794	0.550	0.781	0.348	0.634	0.435	0.706
PSNR ↑	17.36	22.20	16.34	18.90	18.42	23.87	13.31	20.55	14.85	21.97

Table 3: Quantitative evaluation (FID, LPIPS, SSIM, PSNR) of solving linear inverse problems on FFHQ validation dataset. Noise model: Gaussian

Method	Inpaint (random)		Inpaint (box)		SR ($\times 4$)		Deblur (motion)		Deblur (gauss)	
	Ours	Paper	Ours	Paper	Ours	Paper	Ours	Paper	Ours	Paper
FID ↓	56.83	21.19	73.88	33.12	51.02	39.35	104.76	39.92	111.03	44.05
LPIPS ↓	0.271	0.212	0.368	0.168	0.248	0.214	0.442	0.242	0.376	0.257
SSIM ↑	0.723	0.851	0.623	0.873	0.729	0.852	0.490	0.859	0.575	0.811
PSNR ↑	19.86	25.23	16.06	22.47	19.90	25.67	13.81	24.92	16.00	24.25

Table 4: Quantitative evaluation (FID, LPIPS, SSIM, PSNR) of solving linear inverse problems on CelebA validation dataset. Compared with the papers implementation on FFHQ. Noise model: Gaussian

Method	Inpaint (random)		Inpaint (box)		SR ($\times 4$)		Deblur (motion)		Deblur (gauss)	
	Ours	Paper (FFHQ)	Ours	Paper (FFHQ)	Ours	Paper (FFHQ)	Ours	Paper (FFHQ)	Ours	Paper (FFHQ)
FID ↓	33.54	21.19	47.70	33.12	28.28	39.35	87.44	39.92	94.94	44.05
LPIPS ↓	0.245	0.212	0.339	0.168	0.218	0.214	0.428	0.242	0.364	0.257
SSIM ↑	0.744	0.851	0.644	0.873	0.755	0.852	0.295	0.859	0.579	0.811
PSNR ↑	20.56	25.23	17.07	22.47	20.85	25.67	8.98	24.92	16.43	24.25

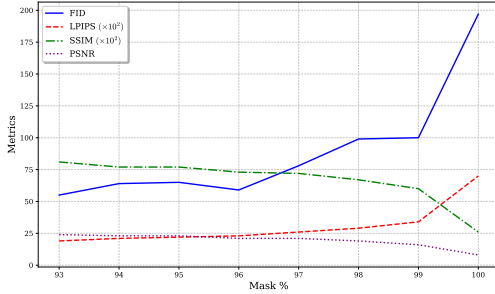
In Figure 1 we observe how the qualitative metrics used, FID, LPIPS, SSIM and PSNR, predictably get worse as the percentage and/or size of the applied mask increases. This is very expected as the mask greatly limits the available information to the network. In Tables 2, 3, and 4 we see the values of the different scoring metrics for our generated images compared to the results from the original paper. We can clearly see how our scores are less impressive than the original authors but are still in the same ballpark. In Table 2 we can visually see the effect of the different measurement models on the CelebA-HQ images and how well the diffusion model is able to reconstruct the distorted images. We found that box-inpainting had the most intriguing results as the network has to fill in a relatively large area of an image, recreating eyes, hair, mouths etc. which it succeeds in fairly well.

Table 5: Quantitative evaluation (FID, LPIPS, SSIM, PSNR) of solving linear inverse problems with on CelebA dataset. Quantitative results was not reported for any dataset in the paper. Noise model: Poisson

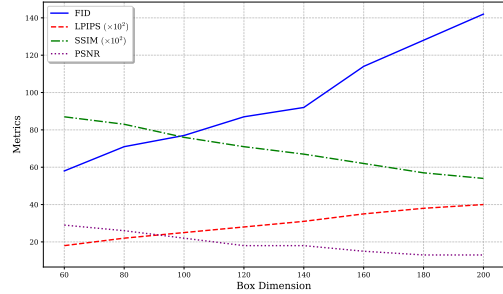
Method	SR ($\times 4$)		Deblur (motion)		Deblur (gauss)	
	Ours	Paper	Ours	Paper	Ours	Paper
FID ↓	38.58	-	86.54	-	96.00	-
LPIPS ↓	0.278	-	0.435	-	0.370	-
SSIM ↑	0.712	-	0.491	-	0.570	-
PSNR ↑	19.62	-	14.06	-	15.47	-

Table 6: LPIPS statistics from solving linear inverse problems on ImageNet, FFHQ and CelebA validation datasets. Noise models are denoted by (GN) for Gaussian noise, and (PN) for Poisson noise. Delta refers to the difference between datapoint and average.

Stats	Dataset	Inpaint (random)	Inpaint (box)	SR ($\times 4$)	Deblur (motion)	Deblur (gauss)
Avg + std						
	ImageNet (GN)	0.361 $\pm$ 0.0058	0.503 $\pm$ 0.0063	0.324 $\pm$ 0.0056	0.586 $\pm$ 0.0062	0.482 $\pm$ 0.0054
	FFHQ (GN)	0.271 $\pm$ 0.0030	0.368 $\pm$ 0.0039	0.248 $\pm$ 0.0028	0.442 $\pm$ 0.0045	0.376 $\pm$ 0.0031
	CelebA (GN)	0.245 $\pm$ 0.0030	0.339 $\pm$ 0.0040	0.218 $\pm$ 0.0028	0.428 $\pm$ 0.0048	0.364 $\pm$ 0.0032
	CelebA (PN)	-	-	0.278 $\pm$ 0.0039	0.435 $\pm$ 0.0049	0.370 $\pm$ 0.0035
Min						
	ImageNet (GN)	0.0933 (Δ 0.2677)	0.1320 (Δ 0.3710)	0.0893 (Δ 0.2347)	0.2478 (Δ 0.3382)	0.2379 (Δ 0.2441)
	FFHQ (GN)	0.1658 (Δ 0.1052)	0.2086 (Δ 0.1594)	0.1587 (Δ 0.0893)	0.2548 (Δ 0.1872)	0.2419 (Δ 0.1341)
	CelebA (GN)	0.1342 (Δ 0.1108)	0.2168 (Δ 0.1222)	0.1165 (Δ 0.1015)	0.1955 (Δ 0.2325)	0.2626 (Δ 0.1014)
	CelebA (PN)	-	-	0.1569 (Δ 0.1211)	0.2262 (Δ 0.2088)	0.2633 (Δ 0.1067)
Max						
	ImageNet (GN)	0.6509 (Δ 0.2899)	0.7530 (Δ 0.2500)	0.6296 (Δ 0.3056)	0.7975 (Δ 0.2115)	0.8203 (Δ 0.3383)
	FFHQ (GN)	0.4459 (Δ 0.1749)	0.5345 (Δ 0.1665)	0.3657 (Δ 0.1177)	0.6319 (Δ 0.1899)	0.5372 (Δ 0.1612)
	CelebA (GN)	0.4033 (Δ 0.1583)	0.5493 (Δ 0.2103)	0.3628 (Δ 0.1448)	0.6654 (Δ 0.2374)	0.5473 (Δ 0.1833)
	CelebA (PN)	-	-	0.5696 (Δ 0.2916)	0.6600 (Δ 0.2250)	0.5572 (Δ 0.1872)



(a) Effect of masked pixels ranging from 93% to 100%.



(b) Effect of increasing box size ranging from 60x60 to 200x200.

Figure 1: Small ablation study on the effect of random inpainting. Data: 10 CelebA images.

Table 7: Quantitative evaluation (FID, LPIPS, SSIM, PSNR) of solving linear and non-linear inverse problems on the CelebA dataset.

(a) Linear				(b) Nonlinear		
Method	Inpaint (random)			Method	Deblur (non-linear)	
	Ours (DPS-DPM)	Ours (DPM)			Ours	Paper (FFHQ)
FID ↓	181.49	33.54		FID ↓	44.67	41.86
LPIPS ↓	0.618	0.245		LPIPS ↓	0.213	0.278
SSIM ↑	0.372	0.744		SSIM ↑	0.616	-
PSNR ↑	16.88	25.23		PSNR ↑	18.75	-



Figure 2: Results on solving inverse problems with Gaussian noise ($\sigma = 0.05$) for different measurement models on the CelebA-HQ dataset

6 Training Time and Resource Analysis

It is stated in the paper that:

"All experiments were performed on a single RTX 2080Ti GPU. FFHQ experiments take about 95 seconds per image (1000 NFE), while ImageNet experiments take about 600 seconds per image (1000 NFE) for reconstruction due to the much larger network size"

The authors do not explicitly specify what datasets they use for their experiments so assuming that they are done on the full validation sets (1k images each) would amount to an approximate inference time of 5.5 days for five measurements operators on the FFHQ dataset, and approximately 35 days for five measurement operators on the ImageNet dataset. We also point out that this is without even taking into account the search for "optimal" parameters and ablation study on the step-size parameter. We ran several experiments utilizing the cloud GPU service Lambda Labs [2024]. Hyperparameter searches and image inferences were run on Nvidia A100 40Gb SXM4 GPUs. FFHQ and Celeba-HQ experiments took 105-120 seconds per image depending on the measurement model. ImageNet experiments took 250-270 seconds depending on the measurement model. Generating 250 images for a single measurement took 8.5 hours for FFHQ and Celeba-HQ and 18.5 hours for ImageNet. Hyperparameter searches and experiments ran for a total time of 312 hours (13 days) with a final bill of 406 dollars or 387 euros. If we had utilized our own Nvidia 1080 Titan-X, hyperparameter searches and experiments would instead have taken 26 days which we decided was unfeasible in the scope of this short project.

Our initial approach was to utilize Google Cloud Platform with academic credits. This was however not possible as we were repeatedly unable to access GPUs on that platform and thus instead resorted to Lambda Labs where we utilized up to 7 Nvidia A100 40Gb GPUs in parallel to speed up the time needed to conduct our experiments.

7 Discussion

7.1 What was easy?

It was easy to acquire the different utilized pre-trained models and corresponding datasets from different Github repositories, HuggingFace and Kaggle. Likewise forking and building upon the Guided diffusion repository was very manageable with good documentation in the repository OpenAI [2024].

7.2 What was difficult?

The main difficulty in the reproduction of the sampling algorithm was with computation; Using our freely available GPUs, each inference took about 7 – 30 minutes. This made the debugging process very hard and time consuming. Another difficulty we faced related to the measurement models was how the measurements were propagated through each step of the reverse process. We initially added noise alongside the measurement operator in the reverse denoising process which is incorrect. This was mainly due to the very limited information from the authors on how the measurement models were actually implemented. There were also several difficulties in reproducing the original paper due to ambiguous statements in the paper. Some examples of this include the authors mentioning that they used the dataset "Imagenet 1k validation" without specifying the year of the one they used. This made it difficult to choose from the many different versions and iterations of the dataset. It was also unclear how many datapoints/image were used in their sampling. Moreover, it was unclear which model they used for the experiments on FFHQ; On page 7 the authors write that *"The diffusion model for FFHQ was trained from scratch using 49k training data (to exclude 1k validation set) for 1M steps"*. However in their implementation details on page 21, they write *"Pre-trained score function for the FFHQ dataset was taken from Choi et al. [2021]"*. The training time on our Nvidia 1080 Titan-X for a simpler network than OpenAI [2024] was 0.5 seconds per iteration with a batchsize of 4, which would roughly correspond to 6 days of training for 1M steps. Our GPU could memory-wise only handle a batchsize of 8 with a training time of 3 seconds per step which accumulated to 35 days for 1M steps. Thus we were unable to train our own FFHQ diffusion model both time and quality

wise. We ultimately chose to use the pre-trained model and added the CelebA-HQ dataset to validate the method on unseen examples of faces.

7.3 Ethical considerations and societal impact

Our project has focused on solving noisy inverse problems, i.e. extracting the original image from a noisy perturbed measurement image. Solving such problems can be highly beneficial in several industries. An example of this being healthcare imaging where such high performing models can be used to assist in improving image quality of various imaging machines (MRI, CT scans, X-rays etc) which is essential when diagnosing a patient. Such applications are highly related to the third United Nations Sustainable Development Goals (UN SDG) regarding good health and well being.

As with most technology there are also of course negative applications that one must be wary of. A clear example of this connected to image reconstruction would be mass-surveillance of a population. Reconstructing images from noisy measurement can just as well be used to extract a persons identity from a noisy image taken by a CCTV camera and used to penalize individuals for certain behaviors and actions. Such applications not only violate an individuals right to live freely but also the foundation of a democratic society.

8 Conclusions

In conclusion we find that we were able to adequately re-implement the *Diffusion Posterior Sampling* algorithm presented in the original paper Chung et al. [2023]. With the aid of cloud GPUs we were able to recreate the experiments for all the different measurement models except for the Fourier phase retrieval with satisfactory reconstruction results. We are very content that DPS and our own code was able to generalize and perform well on the additional CelebA-HQ dataset as well. Moreover, we are able to extend DPS to fast inference ($\approx 6s$) using the fast DPM-Solver Lu et al. [2024], which we believe can be interesting to explore further.

9 Self-Assessment

We have managed to reproduce all major experiments from the original paper to a satisfactory level independently of the authors code repository. Unfortunately we were not able to get identical evaluation values as the paper for the different FID, LPIPS etc scores. Nonetheless we feel that our different scores are still reasonable and in the same order of magnitude as the original paper. Likewise we extended the experiments to the new dataset CelebA-HQ along with ablation studies regarding the different masking techniques of the Inpainting measurement models. We also increase the statistical accuracy of the LPIPS scores by calculating standard deviations for the different datasets which hadn't been done in the original paper. This greatly improves the transparency and insight into the performance of the model. Thus all things considered we believe that this project meets the outlined requirements from our project proposal and that it should receive an A.

References

- Anonymous. Rethinking diffusion posterior sampling: From conditional score estimator to maximizing a posterior. In *Submitted to The Thirteenth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=GcvLoq0oXL>. under review.
- Levi Borodenko. Motion blur, 2024. URL <https://github.com/LeviBorodenko/motionblur>. Accessed: 2024-12-14.
- Jooyoung Choi, Sungwon Kim, Yonghyun Jeong, Youngjune Gwon, and Sungroh Yoon. Ilvr: Conditioning method for denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 14367–14376, October 2021.

- Hyungjin Chung, Jeongsol Kim, Michael Thompson Mccann, Marc Louis Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=OnD9zGAGT0k>.
- Julius O. Smith III. Zero padding theorem (spectral interpolation), 2007. URL https://ccrma.stanford.edu/~jos/dft/Zero_Padding_Theorem_Spectral.html. Available online: https://ccrma.stanford.edu/~jos/dft/Zero_Padding_Theorem_Spectral.html.
- Lambda Labs. Lambda labs: Ai infrastructure and deep learning solutions, 2024. URL <https://lambdalabs.com/>. Accessed: 2024-12-14.
- Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver: a fast ode solver for diffusion probabilistic model sampling in around 10 steps. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9781713871088.
- OpenAI. Guided diffusion, 2024. URL <https://github.com/openai/guided-diffusion>. Accessed: 2024-12-14.
- Maximilian Seitzer. pytorch-fid: FID Score for PyTorch. <https://github.com/mseitzer/pytorch-fid>, August 2020. Version 0.3.0.
- PyTorch-Ignite Team. ignite.metrics.psnr. <https://pytorch.org/ignite/generated/ignite.metrics.PSNR.html>, 2024a. Accessed: 2024-12-14.
- PyTorch-Ignite Team. ignite.metrics.ssim. <https://pytorch.org/ignite/generated/ignite.metrics.SSIM.html>, 2024b. Accessed: 2024-12-14.
- Phong Tran, Anh Tran, Quynh Phung, and Minh Hoai. Explore image deblurring via encoded blur kernel space. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

Appendix A: Experiments with DPM-Solver

Here we summarize our findings on the small study we perform by integrating the fast DPM-Solver for sampling from diffusion models Lu et al. [2024]. In particular we are interested in operating in the extremely low NFE, to explore the feasibility of running the DPM-Sampler in time-critical applications. We run the following experiment,

- *Measurement Operator*: Random Inpainting
- *Noise type* = Gaussian additive noise with $\sigma = 0.05$
- *Dataset*: celebA-hq 256×256 , 250 images
- *Score function*: Pretrained UNet Choi et al. [2021].
- *NFE*: 5
- *step size*: $\zeta_i = 115 / \|y - \mathcal{A}(\hat{x}_0)\|_2$

Running the inference with the setup outlined above takes ≈ 6 seconds on Mac M1, using MPS acceleration. The outputs have high variance, but in some cases we are able to see a near real-time adequate reconstruction of noisy measurements. Some examples can be seen in Figure 3 and the scores can be seen in Table 7 (a).

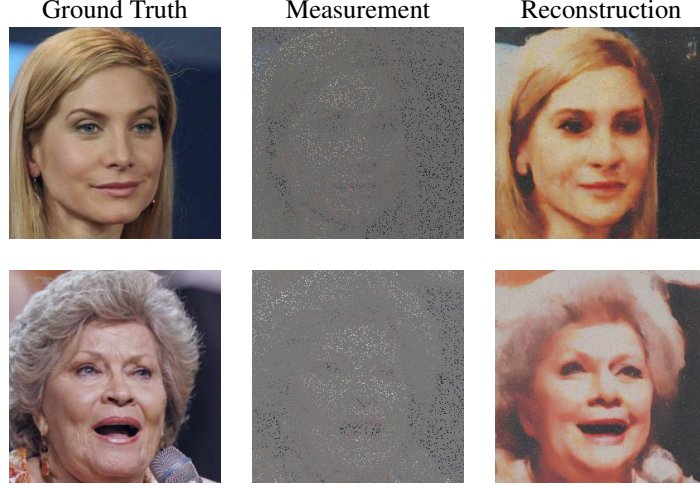


Figure 3: Reconstructions with DPM-Solver with DPS for $\sigma = 0.05$, 92% masked pixels and $NFE = 5$.

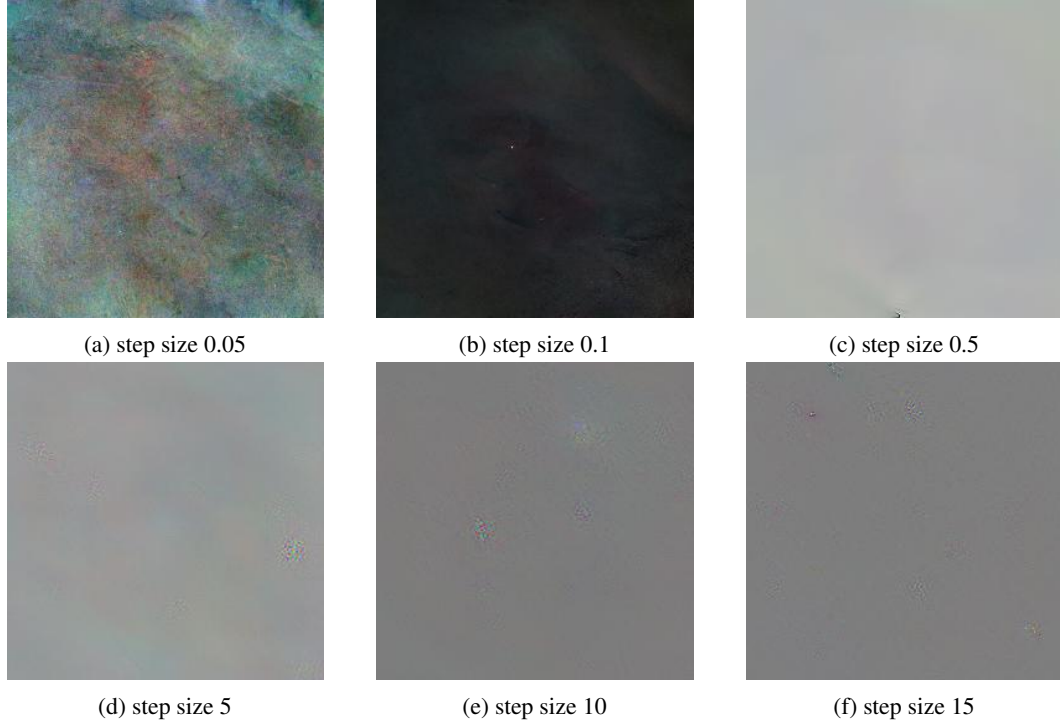


Figure 4: Reconstructions with the Fourier Phase Retrieval measurement model for different step sizes

Appendix B: Issues with PhaseRetrieval measurement

We were unable to reconstruct images based on discrete Fourier transform as the original authors did in their paper as can be observed in Figure 4. Our reconstructions for small step sizes resulted in a colorful mist meanwhile for larger step sizes the reconstructed images simply turned gray. The original authors in the paper Chung et al. [2023] are not very informative on how they implemented the Fourier measurement and reconstruction which resulted in us struggling with recreating this specific measurement. As we were not able to implement the Fourier phase retrieval in a productive way (Figure 4) we chose to exclude this particular measurement operator from our experiments.